SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Efficient Data Access and Storage Optimization for PET/CT Imaging Data

Çelik Yücel

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Efficient Data Access and Storage Optimization for PET/CT Imaging Data

Effizienter Datenzugriff und Speicheroptimierung für PET/CT-Bildgebungsdaten

Author:	Çelik Yücel
Supervisor:	Prof. Dr. Julia Schnabel
Advisor:	Prof. Dr. Stephan Nekolla
Submission Date:	2 March 2026

I confirm that this bachelor's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 2 March 2026 Çelik Yücel

Acknowledgments

I would like to thank Prof. Dr. Julia Schnabel and the chair of Computational Imaging and AI in Medicine for providing the academic framework and institutional setting in which this thesis was conducted. The research environment, with its emphasis on methodological rigor and reproducibility, shaped the broader context of this work. I am particularly grateful to Prof. Dr. Stephan Nekolla for supporting the practical application context of this project within PET/CT imaging research and enabling access to relevant datasets. His perspective on clinical workflows and the relevance of metadata helped ground the system design decisions in real-world imaging practice. Finally, I would like to express my sincere gratitude to my family and friends for their continuous support, patience, and encouragement through my studies. Their trust and motivation made it possible to complete this thesis with focus and determination

Abstract

Medical imaging repositories are large, heterogeneous, and operationally complex. In research and quality assurance (QA) workflows, scalable access to Digital Imaging and Communications in Medicine (DICOM) metadata is often more important than pixel rendering, yet typical tooling remains fragmented between ad hoc scripts, file-level utilities, and heavyweight enterprise systems. This thesis presents a complete DICOM Metadata Browser that transforms heterogeneous DICOM file collections into compact, queryable databanks with integrated quality analytics.

The implemented system combines robust DICOM discovery, metadata-only extraction, preservation of private tags, selection of representative series, and an interactive web user interface (UI). It supports mixed extension conventions (`.dcm`, `.ima`, extensionless files), nested scan directories, single-file ingestion, ZIP archive inputs, deduplication, and multilingual browsing (English and German). The storage layer uses SQLite with indexing and performance-oriented settings, augmented by a dedicated private-tag table that preserves creator information, value fingerprints, and classifications. Siemens Common Syngo Architecture (CSA) payloads are parsed and persisted as structured JSON with hashes for consistency checks.

Evaluating on four local datasets shows fast local processing and strong storage reduction. Measured end-to-end processing times range from 1.44 s to 13.14 s in the test environment. Database size reduction compared with the processed scan folders ranges from 19.2x to 147.3x. Integrated quality checks identify completeness gaps and temporal anomalies, including negative injection-to-acquisition delays in one subset. These results show that a local-first, pragmatically engineered metadata platform can provide meaningful operational value for imaging research and QA without requiring heavy infrastructure.

Contents

1. Introduction

Digital imaging research increasingly depends on metadata quality, not only on image visibility. In contemporary DICOM repositories, researchers and developers must answer questions about protocol consistency, temporal plausibility, dose documentation, device provenance, and cohort comparability before they can trust any downstream model or quantitative analysis. In practice, these questions are often addressed through ad hoc scripts, manual spot checks in viewers, and repeated one-off exports. This workflow style is fragile, difficult to reproduce, and inefficient when datasets change over time.

This thesis presents the design and implementation of a full DICOM Metadata Browser as a coherent systems-engineering response to that workflow gap. The central idea is to operationalize metadata work as a persistent pipeline with explicit architecture, storage, validation, and exploration interfaces. Instead of repeatedly rebuilding one-off script outputs, the system provides a stable metadata layer that supports iterative inspection, filtering, quality checks, and export.

The scientific contribution is therefore not a new DICOM parser algorithm, but a robust integration architecture that unifies heterogeneous file discovery, metadata normalization, persistent, queryable storage, and integrated QA analytics into a single, reproducible artifact. In practical terms, the novelty lies in combining these capabilities under local-first constraints while preserving traceability from dashboard-level anomalies back to concrete series-level metadata records.

The project was developed in the context of a bachelor's thesis and therefore had two explicit constraints. First. the solution had to run reliably in local environments without mandatory external infrastructure. Second, the system had to remain maintainable and understandable within a realistic academic development timeline. These constraints influenced all major design decisions, including the use of Python, pydicom, SQLite, and Flask, the preference for modularized pipeline stages, and the emphasis on interpretable quality checks.

Although the evaluated data originates from medical imaging, the thesis contribution is fundamentally computer-science oriented: architecture design, pipeline orchestration, data modeling, indexing, concurrency control, validation logic, and reproducible tooling. Clinical metadata examples are used as a demanding application domain for validation, not as the primary research method.

The remainder of this thesis is organized as follows. The next chapters formalize the problem context and derive concrete requirements from observed dataset behavior. The architecture and implementation chapters then explain the processing pipeline in depth, including discovery logic, metadata extraction, private-tag handling, storage design, and web-based exploration. The evaluation chapters present measured runtime and storage behavior on local datasets and analyze quality signals detected by the integrate validation

layer. The discussion and conclusion chapters summarize contributions, limitations, and technically grounded future work.

2. Problem Context and Motivation

The practical challenge addressed by this work is not the absence of DICOM parsing libraries. Reliable low-level parsers already exist and are widely used. The challenge is that many research workflows still lack a coherent system that integrates ingestion, normalization, quality evaluation, and exploration into a single reproducible process.

Real-world repositories are rarely homogeneous. A single dataset can contain standard `.dcm` files, `.ima` variants, uppercase extension variants, extensionless files, and non-DICOM artifacts introduced by archiving or operating-system metadata conversions. Transfer pipelines often package data as ZIP or 7Z archives. Institutional preprocessing can modify temporal fields, and anonymization workflows can preserve visual content while unintentionally introducing metadata inconsistencies. If a pipeline is brittle at discovery time or overly strict at parse time, substantial portions of data can be silently dropped.

The second major problem is that script-based extraction tends to produce static, flat files with limited life-cycle support. If users need new fields, changed filters, additional checks, or updated cohorts, they often rerun the entire process and produce another disconnected export. This fragments data lineage and increases the risk of inconsistent assumptions being mixed across analyses.

The third problem is methodological. In many projects, metadata completeness and temporal plausibility are evaluated informally and late in the process. By the time inconsistencies are discovered, downstream analyses may already depend on incorrect assumptions. A better workflow surfaces quality signals early and continuously, and it provides traceable links from quality findings to concrete source records.

Taken together, these observations motivate the system goal of this thesis: to create an integrated, local-first metadata platform that can robustly ingest heterogeneous DICOM repositories, preserve rich metadata context, and expose quality information in a reusable and interpretable form.

2.1. Research Questions, Objectives, and Scope

This thesis is structured around four core research questions. It first investigates whether robust, heterogeneous DICOM discovery can be implemented to improve practical recall without introducing unacceptable false positives. It then examines whether metadata-only extraction, combined with local relational storage, provides sufficient performance and stability for responsive, repeatable workflows on realistic local datasets. A third focus concerns the design of a compact yet expressive quality layer capable of identifying actionable metadata issues, particularly in temporal fields. Finally, the thesis evaluates whether these

technical capabilities can be integrated into a user-facing interface that reduces reliance on one-off scripts and ad hoc processing.

To address these questions, the project pursued six concrete objectives. The system was designed to support multiple input forms, including directories, nested subdirectories, single files, and compressed archives. It performs broad metadata extraction without loading pixel arrays, while preserving private tags and selected vendor payloads instead of discarding them. Extracted records are persisted in a queryable databank with practical indexing and migration support. The platform further enables interactive search, filtering, and export workflows, and integrates quality analytics directly into the same operational interface used for exploration.

The scope of this work is deliberately bounded. The system is not intended to replace clinical Picture Archiving and Communication System (PACS) infrastructure, nor does it function as a diagnostic decision-support tool. It does not attempt full semantic decoding of all private tags across vendors and software versions, and the current implementation does not include enterprise-grade multi-user access control. Instead, the project establishes a robust local research foundation that can be extended and operationalized in future work.

2.2. Technical Background

DICOM is a metadata-rich standard in which each data element consists of a tag, a value representation, and an encoded value[1]. In repository-level workflows, these elements align naturally with patient-, study-, series-, and instance-level contexts. Physical storage conventions, however, do not always preserve this conceptual hierarchy transparently for analysts. Study- and series-level attributes are frequently repeated across instances, and practical pipelines must transform this raw redundancy into structures that are suitable for querying and cohort-level analysis.

During implementation and debugging, interactive tag-browsing tools can accelerate exploratory inspection of element definitions and common tag names[2]. In this work, such utilities serve solely as convenience aids; normative assumptions and design decisions are grounded exclusively in the official DICOM standard[1].

A central technical premise of this thesis is that many metadata workflows do not require pixel data. Pixel arrays dominate file size and I/O cost, whereas fields relevant for cohort filtering, temporal validation, and quality checks reside in metadata headers. Metadata-only extraction, therefore, reduces both runtime and memory footprint without sacrificing analytic utility. This view aligns with the quantitative imaging and AI data-preparation literature, which emphasizes the need for structured metadata curation as a prerequisite for robust downstream analysis[3, 4].

Temporal metadata plays a vital role in PET and nuclear medicine contexts. Injection times, acquisition timestamps, and related date fields are routinely used to derive associated quantities and protocol consistency checks. Even when not directly incorporated into final clinical interpretation, these fields remain essential for workflow validation and reproducibility.

Private tags introduce additional complexity. By design, they are vendor- or pipeline-

specific, and their semantics are not uniformly documented. Nevertheless, they often contain reconstruction parameters, provenance details, and processing context that are operationally valuable. A metadata platform oriented toward research should therefore preserve and expose such payloads rather than discarding them prematurely. Imaging informatics literature on ETL-focused pipelines similarly emphasizes the transition from viewer-centric systems to analytics-oriented metadata architectures[5]

2.2.1. Formal Redundancy and Complexity Model

Let N denote the number of DICOM instances, R the number of series, and S the number of studies, with $S \leq R \leq N$. In flat instance-level storage, study- and series-level metadata are duplicated per instance, causing storage for repeated attributes to scale as $O(N)$. Under hierarchical normalization, these attributes are stored once per logical level, yielding storage complexity $O(S + R + N_{inst})$, where N_{inst} represents truly instance-specific fields.

To illustrate this difference quantitatively, consider a single study with 10 series and 500 instances per series ($N = 5000$). If a study-level block of 1 kB and a series-level block of 0.5 kB are duplicated at the instance level, flat redundancy overhead is approximately

$$5000 \cdot (1.0 + 0.5)\,\text{kB} = 7500\,\text{kB}.$$

With hierarchical normalization, the same information requires only

$$1 \cdot 1.0\,\text{kB} + 10 \cdot 0.5\,\text{kB} = 6.0\,\text{kB}.$$

Even before excluding the pixel payloads, this simplified example yields a redundancy ratio of approximately $7500/6 \approx 1250\times$, demonstrating the structural advantage of hierarchy-aware persistence.

For a more concrete work case, assume 1 study, 10 series, and 300 instances per series ($N = 3000$) with 40 repeated attributes per instance. Suppose 20 attributes are study-level and 20 are series-level, and each stored attribute value (including name, value, and overhead) requires 32 bytes. In flat storage, all 40 attributes are persisted per instance. In normalized storage, study-level attributes are stored once per study, and series-level attributes are stored once per series.

Table 2.1.: Worked numeric redundancy example (flat vs normalized repeated metadata).

Quantity	Flat model	Normalized model
Repeated attribute values stored	120,000	220
Bytes for repeated attributes	3,840,000	7,040
Flat/normalized ratio (bytes)	$3{,}840{,}000/7{,}040 \approx 545\times$	

2.3. Related Work and Positioning

2.3.1. The DICOM Standard and Structured Medical Imaging Data

The Digital Imaging and Communications in Medicine (DICOM) standard underpins contemporary medical imaging workflows. Beyond defining transport and storage conventions, it specifies a hierarchical information model comprising patient, study, series, and instance entities, along with structured metadata modules and globally unique identifiers [1]. Pianykh's practical DICOM guide underscores both the expressive power of this model and the implementation complexity encountered in operational systems [6].

DICOM also supports advanced quantitative objects, including segmentation and structured reporting. Fedorov et al. demonstrate that these capabilities enable standards-compliant quantitative imaging pipelines in PET/CT research [3]. Their work simultaneously highlights the effort required to maintain encoding discipline and validation consistency in real deployments.

While DICOM provides a rich structural foundation, it does not enforce cross-dataset normalization, consistency auditing, or relational abstraction. Clinical systems typically prioritize archival robustness and interoperability. The present thesis addresses the complementary need for a research-oriented relational metadata layer focused on integrity, deduplication, and scalable analysis.

2.3.2. Enterprise Imaging Platforms and Research PACS

Enterprise imaging systems and research PACS platforms illustrate the architectural separation between operational imaging workflows and analytics-oriented processing. REMIX integrates PACS-adjacent services, de-identification modules, and research workflows within a broad institutional stack [7]. Orthanc offers a lightweight, vendor-neutral archive with API access and plugin extensibility, suited to research environments [8].

These platforms provide essential infrastructure for storage and retrieval. However, their primary focus remains interoperability and system integration. They do not inherently provide cross-dataset relational normalization, deduplication-oriented modeling, or systematic temporal anomaly analytics. The system developed in this thesis is therefore positioned as a metadata intelligence layer operating above or alongside archival infrastructure rather than as a replacement for PACS or VNA systems.

2.3.3. Metadata Quality Assurance and Protocol Compliance

Metadata integrity is central to quantitative imaging workflows, particularly in nuclear medicine. Q-Bot demonstrates that automated DICOM metadata monitoring can detect protocol non-compliance and field inconsistencies in routine practice [9]. This confirms that proactive metadata QA has operational value beyond retrospective auditing.

Tsave et al. propose a multi-dimensional data-quality framework encompassing completeness, validity, consistency, integrity, and fairness [10]. Their framework reinforces the premise that data quality must be systematically assessed prior to AI modeling or statistical analysis.

Building on these perspectives, the present thesis integrates QA computation directly into the ingestion and exploration workflow. Deduplication-aware storage, temporal anomaly detection, and private-tag visibility are treated as first-class components of routine metadata processing rather than auxiliary steps.

2.3.4. Privacy, De-Identification, and Re-Identification Risks

The reuse of imaging data for research requires robust de-identification, yet both metadata and pixel-level channels remain potential vectors for re-identification. The Medical Image De-Identification Benchmark Challenge illustrates the technical difficulty of achieving reliable anonymization at scale [11]. Complementary evidence from the New England Journal of Medicine demonstrates that facial reconstruction from cranial MRI can enable re-identification against public photographs [12].

This thesis does not claim full pixel-level anonymization. Instead, it strengthens metadata-side transparency and integrity checking by preserving private-tag evidence, exposing temporal inconsistencies, and enabling auditable review before downstream analysis.

2.3.5. AI Data Preparation and ETL Pipelines

AI-driven imaging research depends on structured preparation pipelines extending beyond archival storage. Diaz et al. describe a lifecycle that spans acquisition, de-identification, curation, storage, and annotation [4]. Their analysis underscores that raw repositories rarely meet analysis-ready standards without deliberate transformation and validation.

Similarly, PDCP demonstrates an ETL-oriented pipeline for extracting research datasets from Orthanc-based environments [5]. These works reinforce the need for transformation layers above PACS APIs. The system presented here aligns with this principle, focusing specifically on metadata normalization and consistency-aware relational modeling for scalable cohort analysis.

2.3.6. Quantitative PET/CT and Standardization

Quantitative PET practice depends on strict protocol adherence and timing accuracy. The EANM FDG PET/CT guideline emphasizes harmonization requirements for reliable standardized uptake value (SUV) interpretation [13]. From a system perspective, temporal metadata fields, therefore, constitute core quality variables rather than peripheral annotations.

The temporal anomalies identified in the evaluated PET/CT subsets in this thesis empirically reinforce this point: without systematic metadata validation, quantitative conclusions may be compromised by hidden protocol inconsistencies.

2.3.7. Relational Modeling and Data Independence

The theoretical foundation for normalization follows Codd's relational model, which formalized data independence and redundancy control for shared data systems [14]. Codd

argued that the logical structure should insulate users from the complexity of physical storage while supporting consistency-preserving operations.

Applied to DICOM repositories, this principle entails translating hierarchical five-level metadata into a normalized relational representation that reduces duplication and improves query reliability across cohorts. The contribution of this thesis lies in synthesizing DICOM-aware extraction with relational data-engineering principles for metadata-centric imaging research.

3. System Design and Requirements

Requirements were derived from iterative interaction with real repository structures. Functional requirements include robust candidate detection, metadata extraction across major tag categories, preservation of private payload, deduplication behavior suitable for reruns, and interactive user workflows for search, filtering, and export.

Non-functional requirements include local deployability, bounded resource use, resilience to imperfect inputs, and maintainability of code structure. Reliability requirements emphasize graceful degradation: unreadable files should be skipped and reported rather than causing a full run failure.

The design principles that emerge from these requirements are consistency, traceability, and interpretability. Consistency requires shared discovery and parsing logic across all tools. Traceability preserves a clear path from aggregated metrics back to the underlying source records. Interpretability ensures that quality signals remain grounded in explicit metadata fields rather than opaque black-box outputs.

3.1. Architecture Overview

The system is structured as a layered pipeline with explicit stage boundaries. Discovery identifies candidate paths. Extraction converts each candidate into a structured metadata record. Storage persists these records in SQLite with indexing and deduplication constraints. A representative-series step then marks one series per study for stable study-level aggregation. Finally, the web layer exposes the databank through a Flask-based interface for browsing, QA, and export.

This separation offers practical benefits: each stage can be verified independently, failure models remain localized, and bottlenecks are easier to attribute. It also enables incremental evolution without destabilizing the whole pipeline. For example, search ranking can be refined in the web layer without touching extraction, and private-tag classification can be extended without changing discovery.

From a software-architecture perspective, the design improves cohesion while controlling coupling. Each stage has a single dominant responsibility (candidate selection, extraction, persistence, aggregation, or presentation), which increases intra-module cohesion. Interfaces between stages are intentionally narrow, primarily typed metadata records and stable identifiers, reducing inter-module coupling and lowering regression risk during feature extension.

The architecture also constrains the growth of complexity in routine runs. Processing N files incurs approximately linear discovery, extraction, and insertion effort $O(N)$. Representative

marking is driven by series grouping and remains linear in the number of observed series. By avoiding nested cross-layer control flow, the pipeline prevents superlinear behavior that would otherwise emerge from repeated rescans and cross-stage feedback loops.

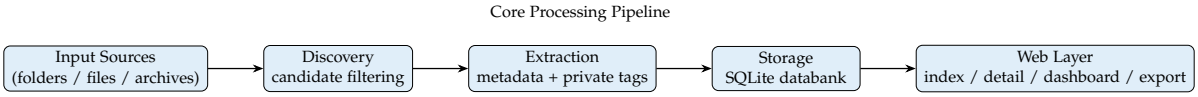


Figure 3.1.: End-to-end processing flow from heterogeneous inputs to interactive QA and export.

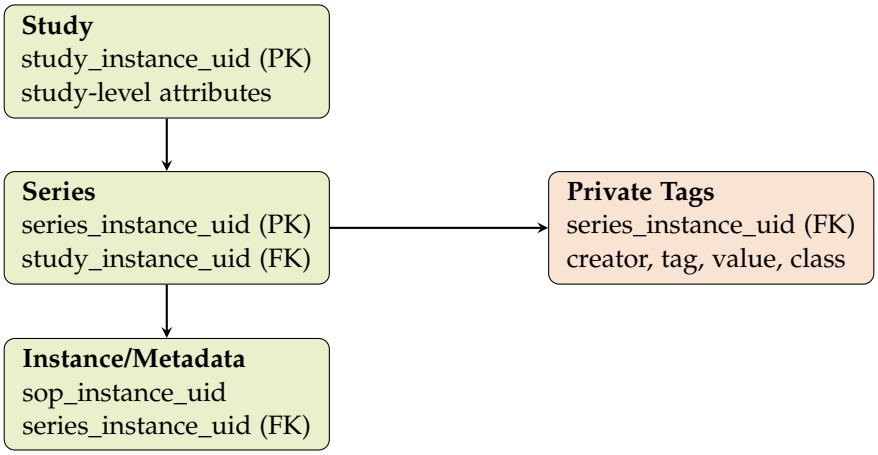


Figure 3.2.: Normalized hierarchy sketch used by the databank model. Repeated study/series fields are stored once per hierarchy level instead of once per instance.

3.2. Input Handling and Orchestration

The orchestration layer accepts heterogeneous inputs. Directories can be processed recursively or treated as separate scan roots depending on flags. For single files, candidate checks are applied immediately. Archives are extracted to a temporary location before processing, after which the extracted content is handled identically to normal folder inputs.

This design reflects common data-delivery practice. Research datasets frequently arrive as compressed bundles with mixed internal structure; integrating archive handling into the pipeline eliminates manual preprocessing steps and reduces user error.

3.3. Discovery Layer

Discovery applies a staged acceptance policy. Files with known DICOM extensions are accepted directly. Files containing the standard DICOM file header marker ("DICM") are also accepted even when no recognized extension is present. For remaining extensionless

files, the system attempts a parser-based fallback to determine whether the file contains valid DICOM metadata. This layered approach balances practical recall and precision when scanning heterogeneous repositories.

Artifact filtering is treated as a first-class responsibility of discovery. Common non-data files (e.g., macOS metadata), archive side structures, and symlink edge cases are excluded to reduce false candidates and avoid unnecessary parser load. The same discovery logic is shared across all processing scripts to ensure consistent file coverage.

3.4. Extraction Layer

Extraction performs metadata-only reads to minimize resource overhead. The field model spans patient-, study-, and series-level attributes as well as manufacturer, acquisition, nuclear medicine, and image descriptors. Conversion helpers normalize common DICOM value types and tolerate malformed optional fields without aborting a full run.

Private-tag extraction is implemented as a parallel path within the same layer. Creator mappings are resolved per dataset, values are decoded conservatively, and value fingerprints are generated for stable tracking. For Siemens Common Syngo Architecture (CSA) payloads, parsing attempts yield structured JSON when feasible and always retain a hash for consistency checks.

3.5. Storage Layer

The storage layer uses SQLite with pragmas and indexes tuned for local analytics. The main table focuses on operational metadata required for browsing and quality logic. High-cardinality private-tag payloads are stored in a dedicated table with uniqueness constraints to prevent repeated duplicates for identical values.

Schema migration is handled during initialization. When older databanks are opened after schema evolution, missing columns are automatically added. This supports iterative development without requiring frequent databank resets.

3.6. Representative-Series Layer

Many studies contain multiple series with overlapping metadata. If all series contribute equally to study-level analytics, dashboards can be biased by overcounting. The representative-series layer, therefore, selects one series per study using a metadata-richness score, prioritizing fields required for uptake-delay and dose checks.

This yields a stable aggregation basis for summaries and QA views, while preserving complete series-level information for drill-down inspection.

3.7. Web and Interaction Layer

The web layer provides study listings, detail views, quality dashboards, and export workflows. Search combines SQL-based retrieval with post-retrieval ranking. Filters support modality, manufacturer, radiopharmaceutical context, and range-based constraints. Export produces grouped field sets and derived values for downstream analysis.

The interaction model supports iterative use: ingest data, inspect quality, refine filters, and export subsets without leaving the same environment.

3.8. Detailed Implementation

The current implementation separates user-facing entry points from reusable internal modules. The root-level entry points are `process_dicom.py`, `extract_one_per_series.py`, and `webui.py`. Reusable application logic resides in the `dicom_browser` package, which contains discovery, extraction, storage, export, dashboard, study-detail, translation, and QA helper modules. This structure keeps operational commands simple while reducing code duplication inside the implementation.

Implementation quality is supported through explicit function boundaries, centralized helper utilities, and defensive handling of common failure modes. The codebase favors clear defaults and robust parsing over assumptions about repository cleanliness.

3.8.1. Script Responsibility Matrix

Because this thesis is evaluated as a software systems project, script-level responsibility is explicitly documented. Table 3.1 summarizes the purpose of each major script and its role in the end-to-end workflow.

This decomposition makes the artifact auditable at code level: each script has a narrow, testable responsibility and can be assessed independently for correctness, performance implications, and contribution to the overall system.

Table 3.1.: Major project entry points and internal modules with their functional purpose.

Component	Purpose	Workflow Role
`dicom_browser.dicom_discovery`	Candidate detection and artifact filtering for heterogeneous file trees.	Shared discovery primitive used by ingestion utilities to ensure consistent file coverage.
`dicom_browser.extract_metadata`	Metadata-only parsing, private-tag extraction, and parallel extraction API.	Core extraction engine used for performance and quality evaluation.
`dicom_browser.store_metadata`	SQLite schema initialization, migration handling, insertion, and dedup constraints.	Persistence layer implementing normalized storage and rerun safety.
`dicom_browser.qa_utils`	Shared helper logic for representative-series scoring, dose calculations, and timing derivation.	Keeps ingestion-time and web-layer QA logic behaviorally aligned.
`dicom_browser.export_utils`	CSV export field metadata, formatting, and anonymization helpers.	Shared export engine for study-level and databank-level CSV workflows.
`dicom_browser.study_service`	Study-detail payload assembly and private-tag enrichment.	Consolidates detail-view logic and keeps route handlers thin.
`dicom_browser.dashboard_service`	Dashboard aggregation, histogram generation, and QA summary computation.	Central analytics layer used by the dashboard route.
`process_dicom`	End-to-end orchestration (discovery, extraction, insertion, representative marking, archive handling).	Main benchmarked processing entry point in Chapter 4.
`extract_one_per_series`	One-instance-per-series subset construction.	Preprocessing utility for filtered-size and raw-vs-filtered benchmark comparisons.
`webui`	Flask application for index/detail/dashboard/export and interactive QA drill-down.	User-facing system component evaluated through workflow screenshots and feature coverage.

3.9. Parallelism and Performance Choices

Parallel extraction uses process-based workers to avoid interpreter-level contention in CPU-bound and I/O-mixed parsing. The worker auto-tunes the samples extraction throughput at startup and selects a practical worker count for the current machine.

Insertion is performed in batch transactions, reducing commit overhead relative to per-file commits. Since measured runs consistently showed extraction dominating insertion, this combination produced a favorable overall profile.

3.9.1. Worker Execution Model and Controls

Worker parallelism is implemented in `extract_metadata.py` using a process pool. Each worker extracts metadata from a subset of candidate files and returns normalized records to the main process for insertion. Parsing, therefore, remains parallel while SQLite access follows a single-writer path.

In default operation (`process_dicom.py` without explicit worker flags), auto-worker tuning is enabled. A bounded candidate sample is processed with several worker-count options (e.g., 1, 2, 4, 8, 16, up to practical CPU/file limits), and the fastest configuration is selected for the full run. This reduces machine-specific guesswork and adapts to local I/O and CPU behavior.

Manual controls are available when deterministic execution is required. `-max-workers N` forces a fixed worker count. `-no-auto-workers` turns off startup tuning and falls back to defaults, where file count and a practical upper cap bound the worker count. For benchmarking, fixed worker counts are preferred to avoid run-to-run variance introduced by tuning samples.

Operationally, throughput increases until parsing saturates storage bandwidth or CPU capacity. Beyond that point, additional workers can increase overhead through scheduling and memory pressure without reducing wall time. The implementation, therefore, treats worker count as a tunable system parameter rather than a fixed constant.

3.10. Deduplication and Incremental Processing

The system supports safe reruns through deduplication and optional path-based skipping. Series-level uniqueness constraints prevent duplicate inflation across repeated processing, while existing-path filtering can skip already-ingested paths when updates are incremental.

This behavior is important for long-lived repositories where new scans arrive periodically, and full reprocessing is unnecessary.

Rerun safety relies on explicit operational assumptions. The current implementation assumes single-writer processing per databank during a run and uses SQLite transactional semantics with uniqueness constraints to guarantee idempotent insertion. Under these assumptions, repeated execution with unchanged inputs converges to a stable databank state rather than accumulating duplicates.

3.11. Quality Logic in Code

Quality computations are intentionally simple and traceable. Delay calculations rely on parsed date and time fields with explicit fallback behavior. Dose-per-kilogram is derived from the injected activity and the available weight. Representative-series scoring combines these computability checks with modality context.

This avoids opaque composite scores and keeps troubleshooting grounded in directly inspectable metadata.

3.12. Comprehensive Functionality Coverage

This section documents the scope of implemented functionality to ensure feature coverage remains explicit and auditable. The system is not only a parser, but a complete local workflow environment spanning ingestion, persistence, exploration, quality assurance, and export.

3.12.1. Pipeline and Storage Functions

Ingestion and Discovery Functions The processing entry points support directories, single files, and compressed archives. Archive ingestion handles ZIP and 7Z packaging, extracts to temporary locations, and processes extracted content with the same discovery logic used for normal folders. Discovery includes extension-based acceptance, signature-aware detection, and parse fallback for extensionless files, while filtering known non-DICOM artifacts. Processing can run recursively by subdirectory or in single-root mode (`-no-subdirs`), enabling both cohort-level and flat-folder usage patterns.

Extraction and Metadata Functions Metadata extraction avoids pixel loading and covers patient, study, series, manufacturer, acquisition, nuclear medicine, and image-geometry attributes. Robust conversion helpers prevent run-level failures when encountering malformed optional fields. Nuclear medicine support includes injection/acquisition timing interpretation, uptake-delay derivation, and dose-per-kg derivation from injected activity and weight context. Private-tag handling includes creator-aware extraction, conservative value decoding, classification heuristics, and persistent storage of decoded values and fingerprints. Siemens CSA handling includes structured JSON capture and hash tracking for consistency checks.

Storage, Migration, and Rerun Safety Functions The databank layer uses SQLite with indexing optimized for search and filtering. The schema includes core metadata and a dedicated private-tag table. Additive migration logic updates older databanks by adding missing columns at initialization time. Rerun safety is implemented via deduplication constraints and optional path-based skipping (`-skip-existing-paths`) for incremental updates. Representative-series marking is executed after ingestion to stabilize study-level aggregation and reduce overcount bias in QA views.

Processing Control and Performance Functions Operational controls include worker parallelism with auto-tuning, manual worker limits (`-max-workers`), optional auto-worker disablement (`-no-auto-workers`), verbose progress output, and timing summaries (`-timing`) that separate extraction and insertion phases. These controls support both reproducible benchmarking and day-to-day processing.

3.12.2. Interface, QA, and Export Functions

Web Interface Functions The web application provides databank-aware navigation and switching, study index browsing, and full study detail inspection. Search combines SQL

retrieval with ranking refinement. Filtering supports modality, manufacturer, and radiopharmaceutical categories, numeric range constraints for uptake and dose, and QA-driven filters for missingness, timing anomalies, dose plausibility issues, composition checks, and QA score thresholds. Study detail views expose expandable per-series metadata blocks, nuclear medicine details, private-tag provenance, and CSA summaries for forensic inspection.

Dashboard and QA Analytics Functions The dashboard provides protocol adherence and QA analytics for representative series. Views include completeness checks, timing-integrity checks, dose-plausibility checks, scanner-landscape summaries, protocol–radiopharmaceutical summaries, derived-object indicators, duplicate/collision indicators, study-composition checks, and QA score distributions. Drill-down links map dashboard findings back to index filters, keeping anomaly detection and root-cause inspection connected within a single workflow.

Export, Administrative, and API Functions Study-level and databank-level CSV export support selectable field sets, grouped export modes, section-based output structure, modality restrictions, derived fields, optional anonymization, and explicit language selection for exported headers. Formatting standardizes dates, times, numeric units, and computed fields to reduce downstream cleanup in analysis notebooks. Administrative actions include databank creation, upload-based ingestion from the UI, and controlled study deletion from the databank. An API endpoint for series-level retrieval supports programmatic access for integration and debugging.

3.12.3. Validation and Demonstration Tooling

The project also includes synthetic data tooling for reproducible demonstrations. Mock generation supports single-file output, structured tree generation, and dashboard-focused cohorts with controlled variation across patient naming, modality, manufacturer, radiopharmaceutical context, delay anomalies, dose-per-kg spread, and missing-value scenarios. This supports repeatable UI demonstrations, regression testing of filters and QA logic, and the generation of thesis-ready figures.

3.12.4. Automated Test and CI Coverage

Beyond manual workflow validation, the implementation includes an automated Python test suite covering discovery logic, extraction helpers, processing helpers, storage integration, anonymized export behavior, and web-level utility functions.

The implemented suite currently comprises 71 passing tests organized under `tests/`. Representative modules are:

- `test_dicom_discovery.py`
- `test_extract_metadata_utils.py`

- `test_process_dicom_helpers.py`
- `test_store_metadata_integration.py`
- `test_study_service.py`
- `test_dashboard_service.py`
- `test_webui_utils.py`
- `test_webui_anonymized_export.py`

Fixtures use synthetic values to avoid exposing real patient identifiers in test artifacts.

Continuous Integration (CI) executes this suite automatically through GitHub Actions (`.github/workflows/tests.yml`) on push and pull-request events. This provides baseline regression protection for core workflow logic, including anonymization controls introduced after the initial evaluation runs.

3.13. Data Model and Query Design

The storage model is pragmatic and optimized for local analytics. It preserves enough structure for meaningful filtering and aggregation while avoiding excessive schema fragmentation. Study and series identifiers provide robust grouping behavior, and indexed columns support common UI and export query paths.

Private payload data is separated into its own table to keep the main table focused and performant. This also improves interpretability: users can inspect private data explicitly when needed without burdening standard queries.

Query design follows two stages. First, SQL retrieves candidates using indexed filters and case-insensitive matching. Second, application-level ranking refines ordering for fuzzy search. This keeps infrastructure simple while improving user-perceived relevance.

3.14. User Workflow and Interface Design

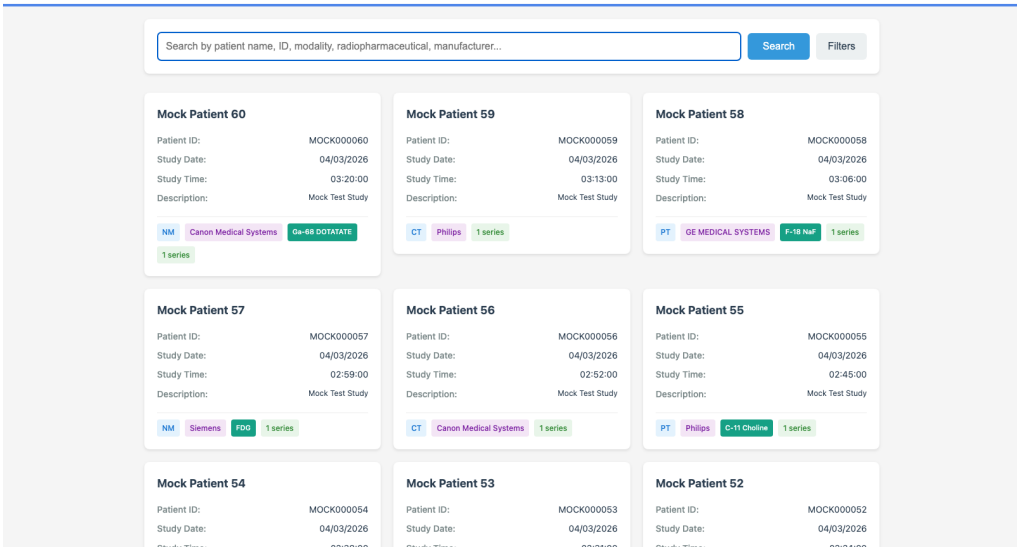
The interface reflects the workflow users typically follow. Sessions begin with broad study exploration, transition to targeted filtering, and end with either record-level inspection or export for external analysis. The dashboard supports this loop as an integrated triage surface rather than a disconnected reporting tool.

A key design principle is progressive detail. High-level pages prioritize compact representative information, while detail pages expose deeper context for individual records. This helps users detect anomalies quickly without being overwhelmed at the start.

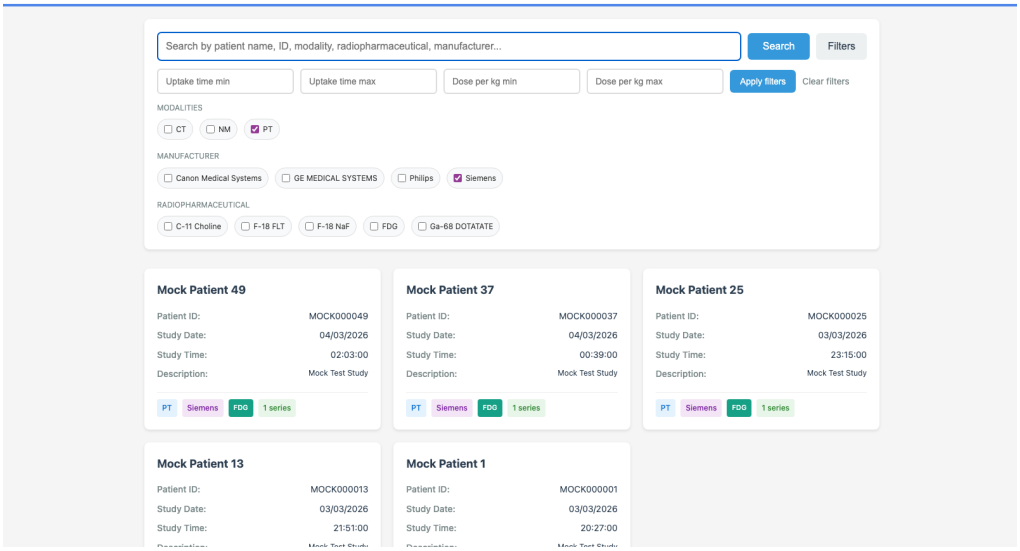
The export subsystem emphasizes interoperability. Users can select meaningful field groups and include derived values commonly required in analysis notebooks, reducing repetitive postprocessing outside the system.

3.15. Visual Interface Walkthrough

To document the concrete interaction surface of the implemented system, this section provides desktop screenshots captured from the `mock_dashboard_demo.db` databank. The index page is the operational entry point for most sessions and supports both broad cohort browsing and targeted subset isolation.



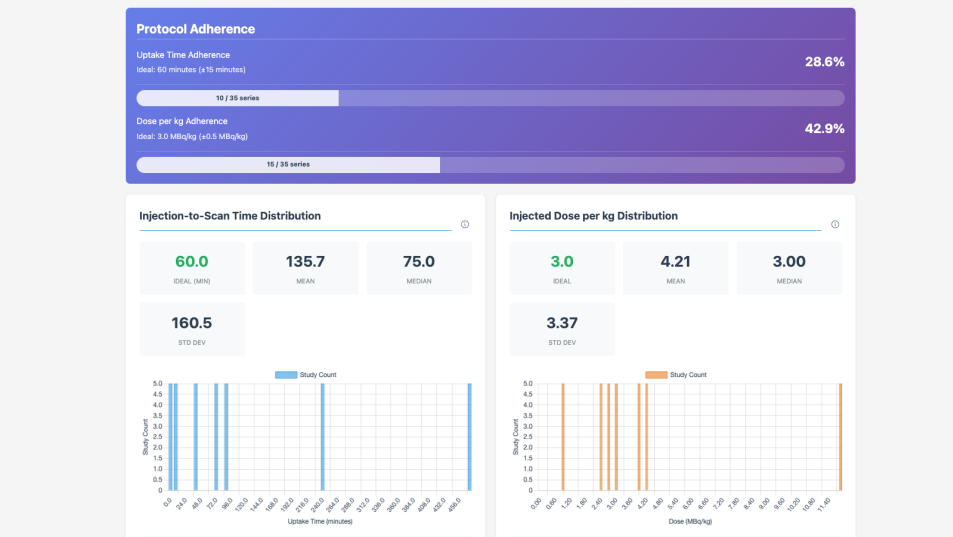
(a) Index overview with global search, filter toggle, and card-based study list.



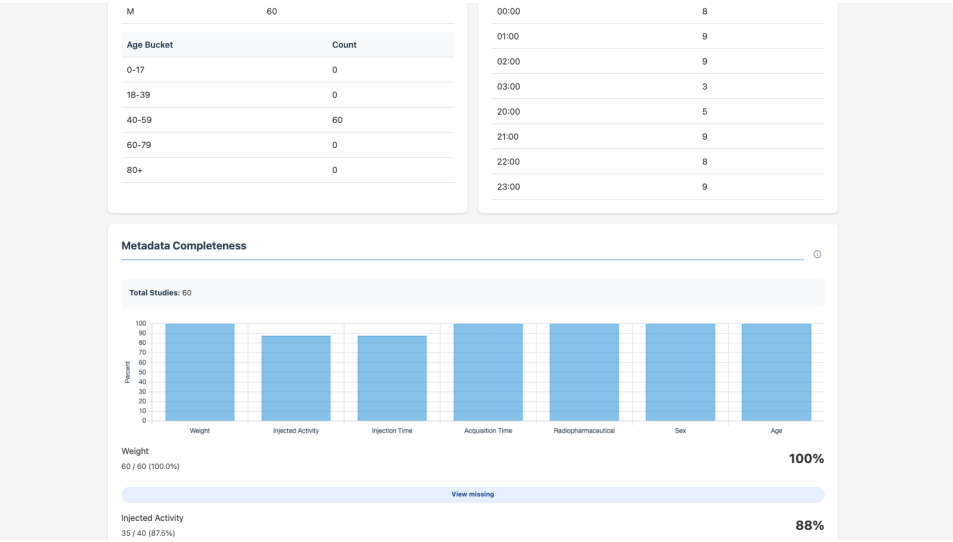
(b) Filtered index view with active modality, manufacturer, radiopharmaceutical, and numeric range constraints.

Figure 3.3.: Index-page workflow views.

The dashboard is used for rapid triage immediately after ingestion. It combines adherence metrics, distribution charts, and QA drill-down links that map directly back to filtered index subsets.



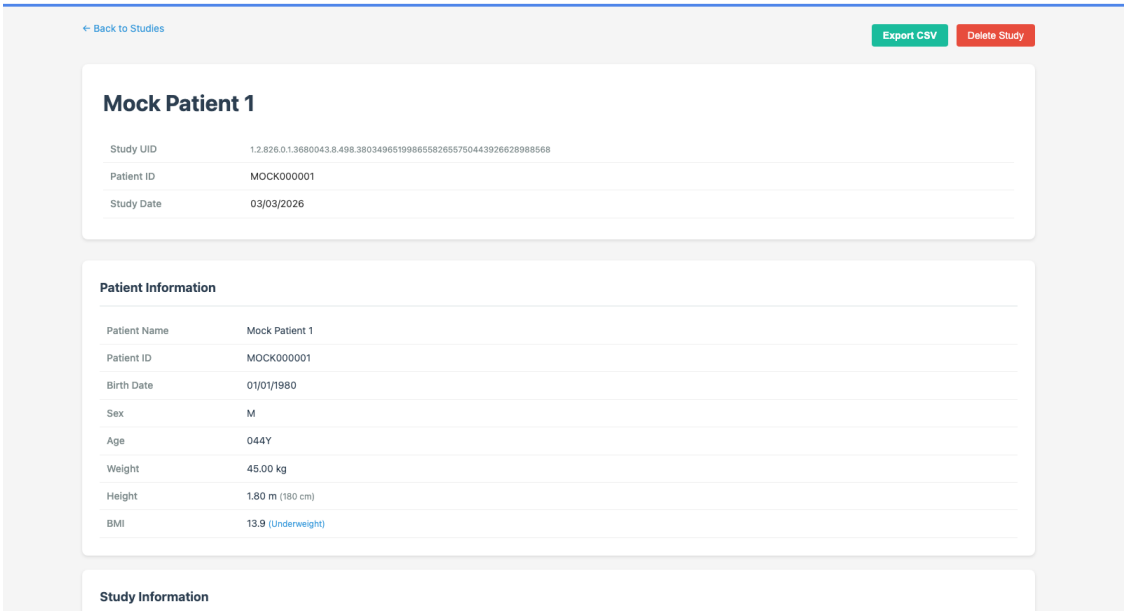
(a) Dashboard overview with protocol-adherence bars and delay/dose distribution summaries.



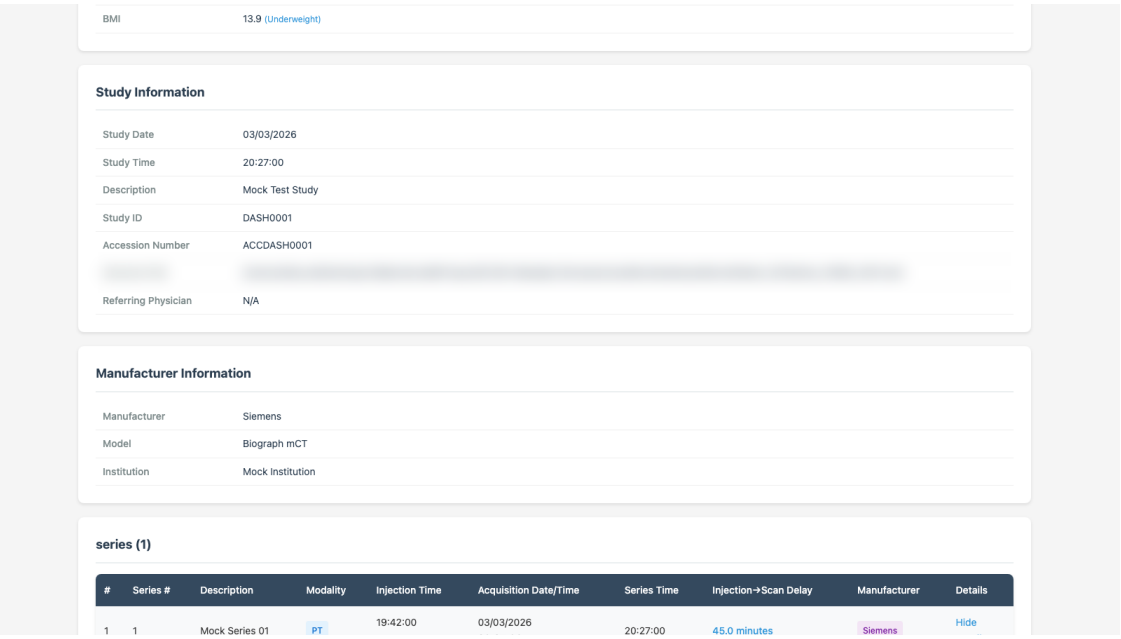
(b) Dashboard QA section showing metadata-completeness bars and per-field missingness drill-down actions.

Figure 3.4.: Dashboard workflow views.

The study detail view supports record-level validation. It exposes full series metadata, expandable per-series panels, private-tag context, and export controls for downstream analysis notebooks and quality reports.

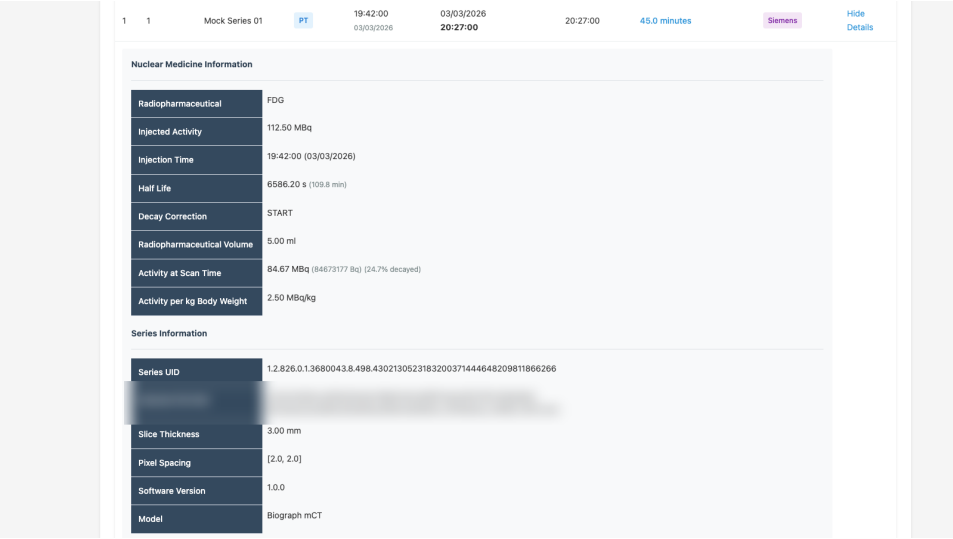


(a) Study detail overview with header actions and patient/study metadata tables.

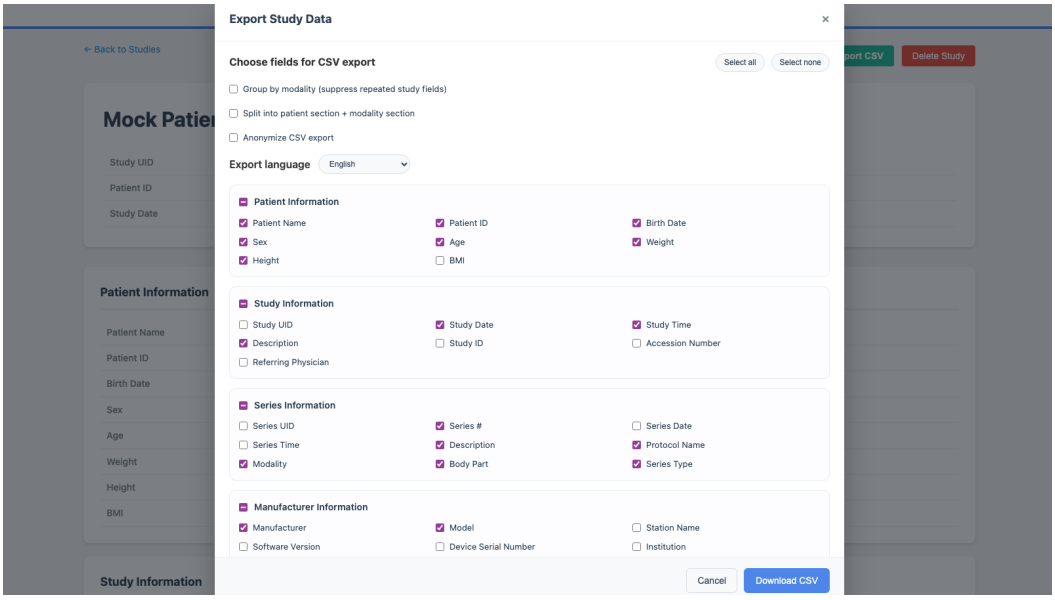


(b) Expanded study-detail view showing the series table with one row opened for inline details.

Figure 3.5.: Study-detail workflow views.



(a) Series details panel showing Nuclear Medicine fields, derived activity values, and technical series attributes.



(b) Export modal with section-based field selection, grouping controls, anonymization toggles, and explicit CSV language selection.

Figure 3.6.: Series inspection and export workflow views.

Absolute local file paths shown in study-level views are intentionally redacted in the figures to protect personal information and local workstation details, while preserving the structure and interpretation of the interface elements.

Several interaction capabilities are described in the report, even when not foregrounded in a specific screenshot. These include databank switching from the top navigation, language switching (German and English), databank creation and upload ingestion, direct study

deletion, CSV export grouped by modality or section, CSV anonymization, export-language selection, and dashboard-to-index drill-down links for missingness, timing, dose plausibility, study composition, and QA score buckets. Together, these functions support a closed loop in which users ingest data, identify anomalies, inspect causal metadata at series level, and export reproducible subsets for external analysis.

4. Evaluation

This chapter evaluates the system as an empirical assessment on four datasets available in the project environment: AnonAxilla, MIRROR_A, PPA, and the public TCIA cohort LMU_PSMA [15]. The measurements cover end-to-end and stage-level runtime, databank size and reduction factors, peak memory (RSS), structural counts, metadata completeness, and temporal-delay anomaly statistics. The evaluation deliberately reports performance together with quality behavior: fast ingestion without usable metadata is unhelpful, while expensive quality logic prevents iterative use.

An additional dataset was originally planned for inclusion, namely the TCIA collection *FDG-PET-CT-Lesions* (`https://www.cancerimagingarchive.net/collection/fdg-pet-ct-lesions/`). Access was unavailable during the project period due to TCIA's controlled-access transition, introduced in 2025 under NIH policy changes (including Guide Notice NOT-OD-25-083 and TCIA's controlled-access hosting sunset timeline). Consequently, only datasets that were accessible and processable locally are reported. For context, TCIA remains a widely used public imaging repository for quantitative imaging research and benchmark development.

4.1. Dataset Structure and Characteristics

Because external readers do not have access to the local input folders used in this thesis, this section summarizes dataset structure, provenance, and analysis-relevant characteristics at a descriptive level.

All datasets follow the standard DICOM hierarchy (Patient → Study → Series → Instance), but differ in scale, preprocessing history, and metadata readiness. Three datasets (AnonAxilla, MIRROR_A, and PPA) were processed as local research subsets, while LMU_PSMA is a public TCIA collection [15]. For comparability, each dataset is profiled by raw size, one-instance-per-series size, resulting databank size, processed instance count, and the number of instances with valid delay computation.

Table 4.1.: Dataset profile summary for readers without direct dataset access.

Dataset	Provenance Category	Raw MB	One-instance-per-series MB	DB MB	Processed Instances
AnonAxilla	Local anonymized subset	219.55	49.81	2.15	130
MIRROR_A	Local anonymized subset	30.89	30.77	1.64	45
PPA	Local anonymized subset	370.40	75.77	6.05	173
LMU_PSMA	Public TCIA collection	117089.00	879.44	6.01	1791

Table 4.1 highlights that scale alone is not a sufficient proxy for metadata readiness:

temporal and radiopharmaceutical field availability varies markedly and directly affects QA interpretability and downstream analytic usability.

4.2. Benchmark Commands

All runs used a consistent command structure:

```
python3 process_dicom.py <INPUT_DIR> <OUTPUT_DB> --no-subdirs --timing
```

The `-no-subdirs` flag ensured consistent directory interpretation, while `-timing` enabled explicit stage-level output. To avoid confounding from auto-worker tuning, worker count was fixed with `-max-workers 8` for raw-vs-filtered comparisons (Table 4.4) and for repeated-run stability measurements (Table 4.6) using `-no-auto-workers -max-workers 8`.

Run Profiles and Scope Mapping

The evaluation uses three run profiles:

- **Filtered-input benchmark runs**: processing one-instance-per-series inputs (provided where available; graphs otherwise). These runs provide runtime decomposition, peak RSS, storage-stage sizes, reduction metrics, and quality counts (Tables 4.2 and 4.3; Figures 4.1a–4.3).
- **Raw-vs-filtered paired runs**: each dataset is processed twice (raw input and filtered input) under fixed workers for direct runtime/RSS and databank-size comparison (Table 4.4; Figure 4.4).
- **Repeated filtered-input runs**: five repeats per dataset under fixed workers for variability estimation (Table 4.6).

4.3. Measured Metrics

Runtime metrics include total elapsed time and stage-level timing (extraction and insertion). Storage metrics compare the raw directory size, the one-instance-per-series subset size, and the resulting databank size. Quality metrics capture field presence counts and delay plausibility outputs. Peak memory usage is reported as resident set size (RSS) measured with `/usr/bin/time -l`.

4.4. Evaluation Results

4.4.1. Performance, Memory, and Storage Outcomes

Filtered-input processing remained practical for iterative workflows: MIRROR_A completed in approximately 0.35 s, AnonAxilla in 0.42 s, and PPA in 0.41 s, while LMU_PSMA required

approximately 5.29 s. In all cases, extraction dominated the runtime, and insertion remained a small component.

Storage reduction was substantial across all datasets. The evaluation reports three size stages (raw folder → one-instance-per-series subset → databank) and derives percentage and x-fold reductions. Reductions are computed as

$$\text{raw} \to \text{one-instance} = \left(1 - \frac{S_{\text{one}}}{S_{\text{raw}}}\right) \cdot 100, \quad \text{one-instance} \to \text{db} = \left(1 - \frac{S_{\text{db}}}{S_{\text{one}}}\right) \cdot 100,$$

$$\text{raw} \to \text{db} = \left(1 - \frac{S_{\text{db}}}{S_{\text{raw}}}\right) \cdot 100.$$

Across the benchmark set, overall raw→db reduction ranges from 94.7% to roughly 99.99%, while the intermediate raw→one-instance-per-series reduction depends strongly on dataset structure.

Table 4.2.: Filtered-input benchmark summary: runtime decomposition, peak RSS, and absolute storage sizes.

Dataset	Time (s)	Extract (s)	Insert (s)	Peak RSS (MiB)	One-instance-per-series MB	DB MB
AnonAxilla	0.42	0.39	0.02	60.8	49.81	2.15
MIRROR_A	0.35	0.33	0.01	57.5	30.77	1.63
PPA	0.41	0.37	0.03	67.9	75.77	6.05
LMU_PSMA	5.29	5.02	0.15	82.6	879.44	6.01

Peak memory remains below 100 MiB for all filtered-input runs despite large differences in raw dataset size and file count. This is consistent with metadata-only parsing and bounded per-file state rather than pixel-array loading.

Table 4.2 also shows that the dominant performance difference is not insertion but extraction time, especially for LMU_PSMA. This supports the later runtime interpretation that front-stage parsing work, not databank writing, is the principal throughput constraint in the current implementation.

Table 4.3.: Storage reduction metrics derived from dataset size stages.

Dataset	Raw→One-per-series (%)	One-per-series→DB (%)	Raw→DB (%)	Raw→DB (x)
AnonAxilla	77.3	95.7	99.0	102.3x
MIRROR_A	0.4	94.7	94.7	18.9x
PPA	79.5	92.0	98.4	61.3x
LMU_PSMA	99.2	99.3	99.99	19486.1x

Table 4.3 makes the structural dependence of reduction explicit. MIRROR_A shows almost no raw-to-one-per-series gain, indicating that its source layout already contains little

instance redundancy, whereas LMU_PSMA benefits enormously from instance reduction before databank storage because repeated image content dominates the raw dataset size.

4.4.2. Raw vs Filtered Effects

Raw-vs-filtered paired runs isolate the impact of redundant instance volume on runtime and memory. For AnonAxilla and PPA, filtering yields moderate runtime and RSS reductions. MIRROR_A changes marginally because raw and filtered inputs are already similar in structure. LMU_PSMA shows the strongest effect: processing the filtered subset reduces runtime from 168.10 s to 5.82 s and peak RSS from about 1.81 GiB to 84.2 MiB, while databank size remains essentially unchanged.

Table 4.4.: Raw vs filtered comparison under fixed workers (`-max-workers 8`).

Dataset	Raw Time (s)	Filtered Time (s)	Raw RSS (MiB)	Filtered RSS (MiB)	Raw DB (MB)	Filtered DB (MB)
AnonAxilla	0.57	0.44	67.7	59.2	2.16	2.16
MIRROR_A	0.35	0.35	59.0	57.8	1.63	1.65
PPA	0.63	0.39	92.5	68.1	6.03	6.04
LMU_PSMA	168.10	5.82	1806.0	84.2	6.03	5.98

Table 4.4 shows that filtering changes runtime and memory much more than final databank size. This is an important practical result: for the current metadata-focused workflow, repeated raw instances impose substantial processing cost even when they contribute comparatively little additional databank information.

4.4.3. Quality and Temporal Integrity Outcomes

Completeness and anomaly behavior varied strongly across datasets. Some cohorts contained rich radiopharmaceutical and timing fields, while others lacked them almost entirely. In this benchmark set, negative-delay anomalies were concentrated in AnonAxilla and MIRROR_A (Table 4.5). Based on dataset provenance and repeated field-pattern inspection, these anomalies are consistent with timestamp side effects introduced during anonymization using Siemens teamplay Privacy Protection App (version 2.1). This reinforces the need for integrated temporal QA: a DICOM object can remain structurally valid while carrying temporally inconsistent metadata after preprocessing.

From a nuclear-medicine perspective, delay checks are clinically meaningful. Injection-to-acquisition timing influences decay correction and quantitative PET interpretation; implausible negative delays or very large positive delays can bias standardized uptake values (SUV) and reduce comparability across cohorts [13]. Integrating this validation during ingestion supports early triage of protocol-inconsistent subsets before model training or statistical analysis.

Table 4.5 further shows that missingness and anomaly behavior are dataset-specific rather than uniform. PPA combines relatively strong timing completeness with no negative delays, while AnonAxilla and MIRROR_A combine smaller valid-delay counts with a high anomaly

Table 4.5.: Selected quality-related counts.

Dataset	Radiopharmaceutical	Dose	Inj. time	Acq. time	Valid delay	Negative	>180 min
AnonAxilla	78	78	78	129	78	58	16
MIRROR_A	20	20	20	38	20	16	4
PPA	96	96	96	173	96	0	0
LMU_PSMA	316	597	597	1194	597	0	3

share, making them less reliable for downstream timing-sensitive PET analyses without additional review.

4.4.4. Repeatability and Runtime Interpretation

Repeated filtered-input runs do not change qualitative conclusions, but they show measurable short-run sensitivity. Relative variance is highest for the smallest jobs (sub-second to low-second runtime), where process startup and filesystem variability contribute proportionally more noise, while the larger LMU_PSMA run remains comparatively stable.

Table 4.6.: Runtime repeatability over five repeated runs per dataset (`-no-auto-workers -max-workers 8`).

Dataset	Mean Runtime (s)	Std. Dev. (s)	CV (%)
AnonAxilla	0.39	0.02	4.2
MIRROR_A	0.35	0.01	2.0
PPA	0.39	0.00	1.1
LMU_PSMA	5.49	0.02	0.3

Table 4.6 indicates that the processing pipeline is stable enough for comparative benchmarking despite short-run noise. The very low coefficient of variation for LMU_PSMA suggests that larger jobs average out startup and filesystem jitter more effectively than the smallest benchmark cases.

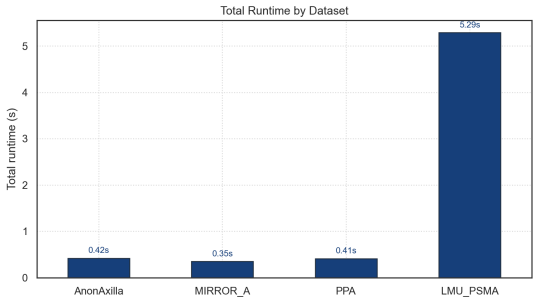
4.4.5. Interpretation of Runtime Composition

Runtime is not determined by instance count alone. Header complexity, private-tag volume, and modality-specific metadata structure affect effective per-instance cost. Extraction dominates because each instance requires repeated file-open operations, DICOM header decoding, private-tag traversal, and field normalization, whereas insertion is a batched write path with relatively small per-row overhead. A useful approximation is

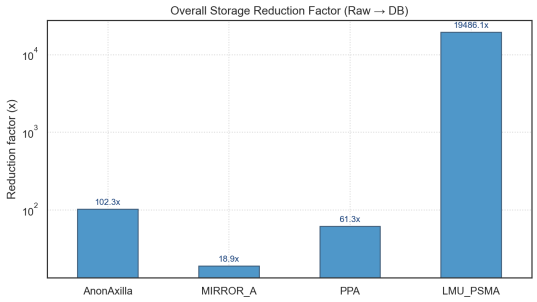
$$T_{\text{extract}}(N) \approx N \cdot (t_{\text{open}} + t_{\text{read}} + t_{\text{parse}} + t_{\text{map}}),$$

which is near-linear in the number of instances N, but with a larger constant factor than insertion. This profile suggests an optimization order driven by the front stage: improve candidate prefilter precision, parser throughput, and read locality first, then tune storage writes.

Figure 4.1a to Figure 4.4 present plots and graphs directly from the benchmark values. Total runtime and memory views use linear scales for direct comparison, while the raw-vs-filtered runtime panel uses a log y-axis to preserve visibility across large gaps.



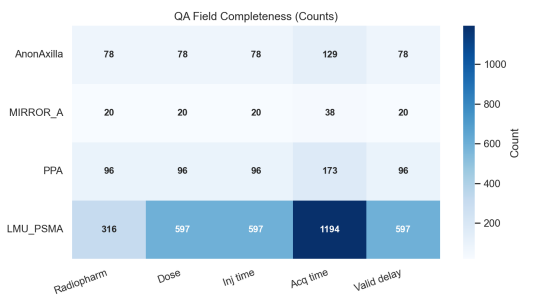
(a) Total runtime by dataset in seconds (linear y-axis).



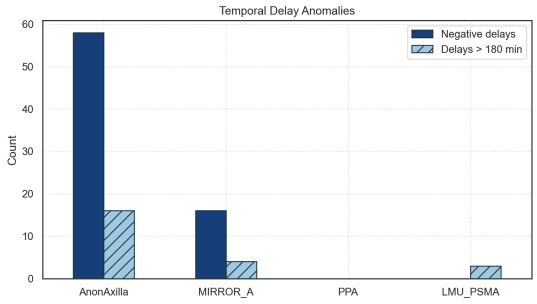
(b) Overall raw-to-databank storage reduction factor by dataset (x-fold, log y-axis).

Figure 4.1.: Performance and storage summary plots. Runtime bars use filtered-input bench- mark runs; storage factors use raw-folder-to-databank-size ratios.

Figure 4.1a makes the runtime imbalance visually immediate: LMU_PSMA is clearly separated from the smaller local subsets, yet even there the filtered-input workflow remains in a range compatible with iterative use. Figure 4.1b complements the table by emphasizing the order-of-magnitude differences in reduction factors; the log scale makes it clear that LMU_PSMA is not merely "somewhat better" but belongs to a different compression regime due to its extreme raw-to-databank size gap.



(a) Heatmap of QA-field completeness counts by dataset (radiopharmaceutical, dose, injection/acquisition time, and valid delay).



(b) Temporal-delay anomaly counts by dataset, separated into negative delays and delays above 180 minutes.

Figure 4.2.: Quality and anomaly summary plots.

Figure 4.2a improves cross-dataset comparison by showing completeness as a pattern rather than as isolated counts: PPA appears comparatively balanced, whereas AnonAxilla and MIRROR_A show visibly sparse timing and radiopharmaceutical coverage. Figure 4.2b reinforces that anomaly burden is concentrated rather than uniformly distributed, which supports the interpretation that preprocessing history, not only dataset size, drives temporal QA risk.

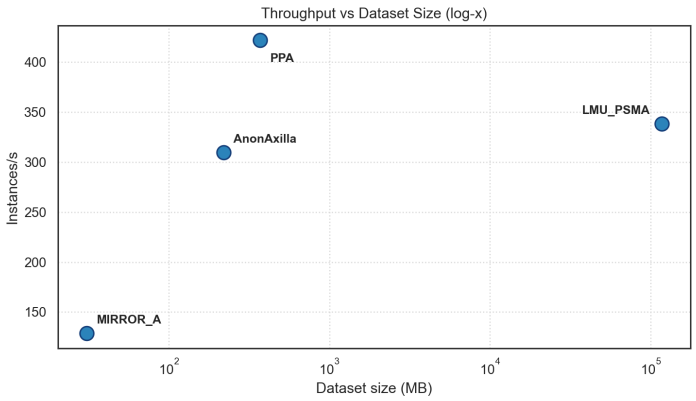


Figure 4.3.: Observed throughput versus raw dataset size (log-scaled x-axis). The scatter plot shows that size alone does not fully explain throughput; metadata composition and parsing characteristics also matter.

Figure 4.3 is useful because it rejects an overly simple "bigger dataset means proportionally slower throughput" interpretation. The spread across points indicates that effective throughput depends not only on raw size, but also on structural redundancy, metadata density, and parser-visible header complexity.

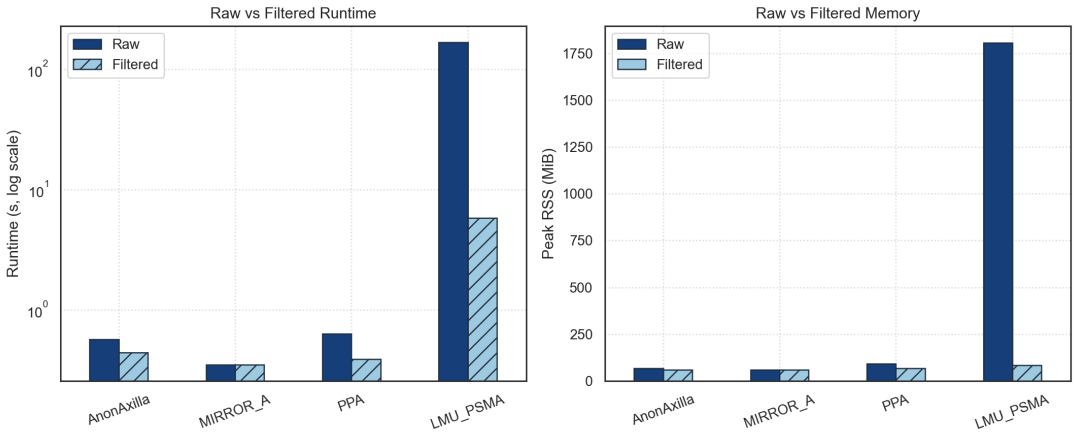


Figure 4.4.: Direct raw-vs-filtered processing comparison for runtime (log-scale y-axis) and peak RSS memory (linear scale). The LMU_PSMA pair shows the greatest improvement after filtering.

Figure 4.4 makes the practical value of instance reduction especially clear for LMU_PSMA: runtime and memory both collapse sharply after filtering, while the databank result changes only marginally. This visual therefore supports the broader claim that repeated instance-level metadata often carries high processing cost but comparatively low incremental value for the current databank-oriented workflow.

4.5. Case Analysis

To complement aggregated metrics, this chapter highlights selected workflow cases.

Archive ingestion reduced manual preprocessing by integrating extraction and cleanup into a single run, even when datasets arrived in mixed formats. Staged discovery improved recall in heterogeneous filename environments compared with extension-only filtering, and the parse fallback for extensionless files was important for vendor-export edge cases.

Temporal QA surfaced problematic subsets immediately after ingestion rather than late in analysis notebooks. In provenance-oriented workflows, preserved private tags provided additional context for comparing scanner and pipeline behavior across cohorts; even without full semantic decoding, creator identifiers and fingerprints improved traceability.

4.6. Measurement Scope and Limits

The evaluation emphasizes throughput, storage behavior, completeness, and anomaly counts because these metrics directly affect practical metadata workflows. Peak RSS provides a coarse view of memory behavior but does not separate parser memory from framework overhead, nor does it capture per-worker memory under alternative parallel settings. This limitation does not affect the runtime and storage findings, but it constrains fine-grained scalability claims for very large file counts. Extending the benchmark harness with stage-level memory profiling is a concrete next step.

5. Discussion

The project outcomes support the thesis that metadata workflows benefit from integrated system design. The implementation demonstrates that local-first architecture can deliver robust ingestion, persistent exploration, and quality-aware analysis support without high operational overhead.

Several trade-offs are explicit. SQLite is highly effective at current scale and simplicity requirements, but larger multi-user settings may eventually require a different backend. Private-tag classification heuristics are useful for operational triage but not a substitute for complete vendor semantic models. Quality thresholds used in delay plausibility checks are practical defaults and should be adapted to the protocol context in future work.

This interpretation aligns with broader data-quality and radiomics workflow literature, which treats metadata quality as a multi-dimensional concern spanning completeness, consistency, temporal validity, and semantic harmonization rather than a single scalar score [10, 7].

From an engineering perspective, the strongest design decision was to consolidate all workflow stages into a single artifact. This reduced duplicated logic and supported reproducible iteration while moving quality checks earlier in the workflow.

5.1. Threats to Validity

Environment-specific benchmarking conditions influence internal validity. Runtime values depend on hardware, storage behavior, and background system load. To reduce this risk, measurements were collected with consistent command patterns and interpreted comparatively rather than as universal constants.

External validity is limited by the diversity of the dataset. Although the evaluated sets differ in size and metadata characteristics, they do not represent all modalities, vendor combinations, or institutional preprocessing pipelines.

Proxy metrics limit construct validity. Field presence is a practical indicator of completeness but not a full guarantee of semantic correctness. Delay anomaly counts are useful signals but not definitive proof of error without domain-specific review. An additional construct validity risk is that preprocessing performed by external de-identification pipelines may alter temporal fields while preserving formal DICOM conformance, potentially shifting anomaly distributions independently of scanner-side acquisition behavior.

Despite these limits, the evaluation remains valid for the stated goal: assessing the practical utility of a local metadata platform in realistic research workflows.

5.2. Limitations

The current implementation has several acknowledged limitations. Automated testing covers core discovery, extraction helper logic, processing helpers, anonymized export behavior, and key storage/web integration paths, but coverage is still incomplete for rare parser edge cases, long-running concurrency scenarios, and large heterogeneous archive stress cases. Private-tag interpretation remains largely heuristic beyond selected vendor payload parsing. Security and access controls are suitable for controlled local contexts but not hardened for broader multi-user deployment.

In addition, while representative-series selection improves dashboard aggregation behavior, certain specialized analyses may still require full series-level or instance-level treatment. The platform preserves enough context to support such extensions, but this is not the current default aggregation model.

A further practical limitation concerns the availability of datasets. Although the FDG-PET-CT-Lesions collection was intended as an additional external benchmark, 2025 constraints on controlled access prevented its inclusion in this project workflow. This constrained cross-dataset comparison breadth and should be considered when interpreting generalization claims.

In addition, the anonymization integrity evaluation in this thesis aligns conceptually with current benchmark-oriented de-identification discourse (e.g., MIDI-B style benchmarking). Still, it has not yet been directly benchmarked against those challenge protocols [11]. The empirical concentration of negative delays in AnonAxilla and MIRROR_A indicates that the Siemens teamplay Privacy Protection App (version 2.1), used for those subsets, likely altered temporal fields in a way that preserved formal DICOM validity while violating clinical time-order plausibility.

According to project-internal supervisor communication, this issue was reported to the vendor, and an updated tool version was subsequently provided. This thesis does not independently verify the vendor-side correction and therefore reports only measurements obtained from the version 2.1 workflow used during the evaluated runs.

Additional technical limits should be made explicit. First, the current aggregation model is optimized for representative static-series analysis and does not yet model dynamic PET frame-level kinetics as a first-class entity. Second, the SQLite operations in this thesis are engineered for single-writer processing per databank run; heavy concurrent multi-writer ingestion would require architectural changes to locking and job orchestration. Third, severe corruption of the unique identifier (UID) hierarchy (e.g., inconsistent Study/Series/Service-Object-Pair (SOP) linkage across files) can reduce grouping reliability and would benefit from dedicated hierarchy-repair heuristics. Fourth, cross-study normalization conflicts can arise when nominally identical protocol labels map to different scanner software behaviors; this currently requires analyst interpretation rather than automatic semantic harmonization.

5.3. Extended Analysis

5.3.1. Extended System Analysis

This chapter deepens the technical analysis of the implemented system by examining how design choices interact with reliability, maintainability, and long-term extensibility. While previous chapters established the architecture and empirical behavior, the present chapter focuses on engineering rationale under realistic constraints.

A recurring insight during development was that metadata pipelines are vulnerable to compounding assumptions. If discovery assumes uniform naming, extraction assumes strict formatting, and storage assumes complete identifiers, then each assumption can amplify errors from the previous stage. The final design, therefore, avoids brittle dependencies and instead uses defensive boundaries with explicit fallback behavior. This strategy increases robustness in heterogeneous repositories at the cost of slightly more code complexity, but the trade-off is favorable for practical use.

Another key insight concerns the stability of workflows over time. In research projects, requirements evolve: new fields become important, quality thresholds are revised, and datasets arrive incrementally. A system optimized for a single fixed run configuration quickly becomes obsolete. By including migration handling, representative re-marking, and rerun-safe insertion behavior, the implementation supports longitudinal use rather than one-time extraction.

Failure Isolation as a Design Objective Failure isolation was treated as a first-class objective. The processing pipeline should continue when individual files are unreadable or malformed, while preserving enough accounting information for diagnosis. In practice, this required local error handling at the file level and the avoidance of global failure propagation from optional fields.

This objective has methodological implications. A pipeline that fails hard on the first malformed file may look cleaner in controlled benchmarks, but is less useful in real repositories. By contrast, a resilient pipeline can produce usable outputs even when inputs are imperfect, allowing analysts to inspect quality distributions rather than only binary pass/fail outcomes.

Schema Evolution and Backward Compatibility Research software often evolves faster than its data artifacts. Databanks created in early versions should remain useful after schema expansion. The migration strategy in this project is additive and conservative: missing columns are added at initialization, and existing data remains intact. This avoids forcing users to regenerate all databanks whenever metadata coverage expands.

Backward compatibility also improves collaborative workflows. Team members can share databanks generated at slightly different times and still reopen them in updated versions of the tool. In academic settings where hardware and schedule constraints vary across contributors, this is a practical advantage.

Interpretability of Quality Signals Quality metrics were intentionally designed for interpretability. Every displayed indicator should map to concrete fields and explicit computation logic. This approach avoids overfitting user trust to opaque scores and supports manual validation when anomalies appear.

Interpretability also supports supervision and review. Advisors can inspect whether a metric is semantically justified and whether thresholds are plausible in context. In contrast, black-box quality outputs are difficult to critique and less useful in scientific reporting.

5.3.2. Deployment and Operationalization Considerations

Although the current implementation targets local research settings, deployment considerations were analyzed to support transition toward broader use. The most important distinction is between single-user exploratory workflows and multi-user operational environments.

In single-user settings, simplicity dominates. Local SQLite storage, direct command invocation, and integrated web browsing provide a productive workflow with minimal overhead. In multi-user settings, additional concerns arise: concurrency management, permission boundaries, auditability, and lifecycle governance of exported subsets.

A practical transition path should preserve existing module boundaries while replacing selected infrastructure components. For example, storage can migrate from SQLite to a client-server database without changing discovery and extraction logic. Similarly, ingestion can move from interactive command execution to background job queues while preserving extraction code paths.

Operational Monitoring Current logs provide basic runtime feedback, but operational deployment benefits from structured monitoring. A future-ready monitoring layer should report ingestion throughput, invalid-file rates, private-tag volume trends, and quality-flag distributions over time. Such monitoring can reveal upstream pipeline changes and silent regressions before they affect analysis outputs.

Backup and Recovery Databank artifacts are compact relative to source repositories, making backup strategies practical. Regular snapshotting of databanks, together with command history and software version metadata, enables recovery of analysis states. This is especially important when quality findings influence inclusion/exclusion decisions in ongoing studies.

Interface Stability For long-running projects, interface stability matters as much as raw capability. Small but frequent UI behavior changes can disrupt analysts. Therefore, future evolution should favor additive enhancements and preserve key interaction patterns for search, filtering, and export.

5.3.3. Data Protection, Ethics, and Governance

Medical metadata workflows require explicit attention to governance even in research settings. De-identification reduces direct identification risk but does not eliminate sensitivity.

Metadata can still encode institutional patterns, temporal information, and private pipeline markers that require controlled handling. From an integrity perspective, governance must also cover the side effects of anonymization tooling itself, because timestamp transformations can silently weaken protocol-level interpretability even when identifying fields are removed correctly.

This thesis does not claim legal compliance certification; instead, it proposes a practical governance posture aligned with research reality. The core principle is minimizing uncontrolled data movement. Users should export only necessary fields, store databanks in controlled locations, and document processing steps that affect data interpretation.

Implementation updates following the core evaluation added two closely related export-governance features to the web interface. First, users can export an anonymized copy of the SQLite databank while replacing selected identifying fields (for example patient name or patient ID) with arbitrary alphanumeric placeholders. Second, CSV export now supports optional export-time anonymization and explicit language selection for the exported headers. Together, these updates improve data minimization and make controlled data sharing more practical without requiring ad hoc post-processing outside the platform.

Risk Categories Governance risks can be grouped into confidentiality, integrity, and reproducibility risks. Confidentiality risk arises when metadata or exports are shared beyond intended boundaries. Integrity risk arises when preprocessing modifies the semantic meaning without clear documentation. Reproducibility risk arises when the command history and the dataset state are not traceable.

The implemented system mitigates parts of these risks through persistent databanks, explicit command workflows, and quality analytics, but additional controls are required for broader deployment. These include role-based access, encrypted storage policies, and auditable export logs.

Ethical Use in Research Contexts Ethical use requires that quality findings are not overinterpreted beyond their methodological scope. For example, detection of temporal anomalies indicates potential inconsistency, not automatic exclusion. Domain experts must review context before drawing strong conclusions. A responsible workflow combines automated indicators with human judgment.

5.3.4. Reproducibility Framework

Reproducibility in this project is treated as a layered property spanning software, data snapshots, and workflow execution. The current baseline is strong for day-to-day research use: explicit command-line runs, persistent databank artifacts, and stable representative-series logic. The next maturity step is automated provenance capture per run, including revision identifiers, command arguments, input snapshot signatures, and quality-metric summaries. In collaborative settings, this should be complemented by shared processing profiles and threshold conventions so equivalent runs produce comparable outputs.

5.3.5. Expanded Evaluation Commentary

The measured runtime profile indicates a clear optimization target: extraction, not insertion. This suggests that future performance work should focus on parser throughput, I/O locality, and candidate prefiltering rather than solely on storage tuning. The existing insertion strategy is already efficient for the current scale.

Storage reduction behavior demonstrates the effectiveness of metadata-first persistence. The observed ratios are expected because pixel arrays dominate source size. However, reduction factors differ across datasets due to modality composition, series structure, and metadata richness. This variation is informative and should be reported rather than normalized away.

Completeness distributions underscore a key methodological point: dataset size does not imply analysis readiness. A large repository with sparse key fields may be less useful for specific tasks than a smaller but complete dataset. Integrated completeness views are therefore not auxiliary features; they are central to study planning.

Temporal anomalies observed in one subset illustrate the value of immediate quality visibility. Without integrated checks, such issues may remain undiscovered until late statistical analysis. Early detection allows targeted review and reduces the risk of invalid downstream assumptions.

Sensitivity to Threshold Choices Quality metrics that depend on thresholds should be interpreted as policy-dependent outputs. The threshold itself is not a universal truth but a configurable boundary for a workflow objective. Future versions should externalize thresholds into profile definitions to improve transparency and comparability.

Longitudinal Evaluation Potential As additional cohorts are processed over time, the system can support longitudinal quality monitoring. Trends in completeness, anomaly rates, and private-tag distributions may reveal changes in institutional pipelines or scanner configuration transitions. This opens a path from one-time evaluation to ongoing metadata observability.

5.4. Operational Outlook and Continuation

5.4.1. Practical Guidance for Thesis and Project Continuation

This section summarizes concrete guidance for continuing the project after thesis submission. A first priority remains test expansion beyond the now-established baseline suite: property-based and fuzz-style tests should target parser robustness, malformed private payloads, and temporal-field edge cases. Integration tests should further cover representative ingestion scenarios with synthetic fixtures and selected real subsets, especially for complex archive and incremental rerun workflows.

The second priority is configurability. Quality thresholds, modality mappings, and representative scoring weights should move into explicit configuration files. This change will improve transparency and reduce the need for code edits for workflow-specific adaptations.

The third priority is documentation structure. Documentation should be separated into a user handbook, a developer guide, and an operations guide. This will improve onboarding and reduce dependence on implicit project memory.

The fourth priority is optional scaling paths. If usage expands to concurrent multi-user patterns, storage and ingestion services can be separated while preserving existing module boundaries. This evolutionary path is feasible because the current architecture is already layered.

5.4.2. Synthesis and Outlook in Research Practice

A central outcome of this thesis is the demonstration that metadata engineering can be operationalized as an iterative scientific workflow rather than a one-time preprocessing script. In traditional script-first workflows, quality checks are often appended after extraction, making them vulnerable to omission. In the presented system, quality checks are structurally integrated with ingestion, storage, and exploration, so quality information becomes part of the default workflow rather than an optional add-on.

This integration has practical consequences for research planning. When researchers begin with an explicit metadata quality profile, they can make earlier and more defensible decisions about cohort composition, exclusion criteria, and analysis feasibility. They can also communicate these decisions more transparently to supervisors and collaborators because the underlying evidence remains queryable in a persistent databank.

Another important synthesis point concerns technical debt and sustainability. Small research tools often become brittle as their use expands. In this project, maintainability was supported by explicit module boundaries, migration-aware storage initialization, and shared discovery logic. These design choices create a stronger base for follow-up work, especially in academic settings where projects are handed over between students.

Beyond immediate engineering utility, the system supports future research on metadata governance, anomaly detection, and protocol harmonization. The persistent databank model makes retrospective analysis easier than repeated flat-file exports and provides a reusable base artifact for follow-up work.

5.4.3. Implications for Future Thesis and Team Work

Future thesis projects can build on this foundation in two complementary directions. One direction is scientific deepening, for example, by studying private-tag semantics systematically or by validating quality thresholds against protocol-specific expert labels. The other direction is systems scaling, for example, by introducing asynchronous ingestion services, role-based access controls, and long-term monitoring dashboards.

Importantly, these directions are not mutually exclusive. A scalable architecture without interpretable quality semantics has limited research value, and highly detailed semantics without operational robustness are difficult to apply in practice. The most impactful continuation combines both.

5.4.4. Final Reflection

This thesis was developed under realistic engineering constraints and therefore emphasizes practical correctness over idealized completeness. The resulting system is not presented as a finished endpoint but as a stable and useful artifact that already improves workflow quality. Its strongest contribution lies in making metadata quality visible, reproducible, and operationally actionable in day-to-day research practice.

5.5. Methodological Positioning

5.5.1. Methodological Rigor and Decision Traceability

This thesis does not claim algorithmic novelty in the sense of a new parsing algorithm or a new anomaly-detection model. The primary innovation is architectural and methodological: integrating discovery, extraction, normalization, validation, persistence, and exploration into one reproducible system artifact with explicit decision traceability.

A common weakness in applied software theses is that engineering decisions are described only as implementation facts without making their methodological consequences explicit. In this work, decision traceability was treated as part of the scientific contribution. Every major technical decision in the pipeline can be connected to a research workflow need and to an observable effect in evaluation output.

For discovery, the decision to combine extension checks, signature checks, and parse fallback was motivated by heterogeneous real repositories and validated by improved practical candidate coverage. For extraction, the decision to avoid pixel loading was motivated by metadata-centric use cases and validated by runtime behavior that supports iterative processing. For storage, the decision to use an indexed SQLite databank with migration handling was motivated by local deployability and longitudinal project use, and validated by stable rerun behavior and compact storage footprints.

For quality analytics, the decision to focus on interpretable metrics rather than opaque composite models was motivated by the need for supervisor review and practical troubleshooting. This decision was validated by the ability to connect anomaly counts directly to known field combinations and by the usefulness of those signals during dataset triage.

This traceability perspective is important because it distinguishes a coherent engineering thesis from a collection of loosely related features. A thesis-quality system should not only work; it should also provide a clear argument for why it was built that way and how observed results support those design choices.

5.5.2. Comparative Positioning Against Typical Research Pipelines

To understand the project's practical contribution, it is useful to compare it with the most common baseline in academic environments: script-based metadata extraction into CSV or JSON, followed by notebook-specific post-processing. That baseline has clear strengths. It

is flexible, easy to start, and familiar to many researchers. However, it also has recurring weaknesses.

First, script baselines often duplicate logic across projects and cohorts. Slightly different discovery rules, naming assumptions, or parsing shortcuts can produce incompatible outputs that are difficult to reconcile. Second, quality checks are often added late and inconsistently, increasing the risk that silent data issues will propagate into statistical analysis. Third, flat exports can become hard to maintain when new questions require reprocessing with different fields or filters.

The system presented in this thesis addresses these weaknesses through persistent, queryable state and integrated quality workflows. Instead of treating each export as a final artifact, it treats the databank as a living metadata layer that can support repeated exploration and evolving analysis questions. This does not eliminate the value of exports; rather, it changes their role from primary storage to interoperability output.

Compared with heavy enterprise systems, the contribution is different. Enterprise solutions typically excel in archival guarantees and broad integration, but they can be cumbersome for custom local analysis loops in small teams. The local-first thesis system reduces operational friction for exploratory research while preserving a clear path for future scaling, if needed.

5.5.3. Positioning Relative to Orthanc and dcm4che

A likely comparison point in defense is Orthanc or dcm4che. These platforms are strong infrastructure components, but their default design center differs from this thesis artifact. Orthanc is primarily a DICOM server ecosystem optimized for interoperable service operation (e.g., storage, query/retrieve, plugin-based extensions). dcm4che is a broad toolkit and server stack with strong standards coverage and integration capabilities. By contrast, the present system targets local-first metadata normalization, relational persistence, and integrated QA analytics for research workflows where rapid cohort triage and anomaly visibility are primary goals.

The practical distinction is workflow orientation. In this thesis system, the discovery of heterogeneous files, representative-series aggregation, QA anomaly logic, and export-ready derived metadata are integrated by default into a single lightweight pipeline. In Orthanc/dcm4che, equivalent behavior is achievable but typically requires additional configuration, plugins, or external analytics layers. Therefore, this work should be interpreted as complementary rather than competitive: it emphasizes metadata QA and reproducible analysis loops over the breadth of server-centric deployment.

6. Conclusion

This thesis presented a complete DICOM Metadata Browser that addresses practical metadata workflow challenges through robust ingestion, metadata-focused extraction, compact persistence, and integrated quality analytics. The system was designed for local deployability, reproducibility, and maintainability, and it was evaluated empirically on multiple datasets.

The results show that the platform provides operationally useful runtimes, substantial storage reduction, and meaningful quality signals. More importantly, it demonstrates that early integrated QA and persistent databanks improve workflow governance compared with repeated script-specific exports. This shift improves both technical workflow stability and methodological rigor.

The thesis therefore proves that a local-first, metadata-centered architecture can remain both computationally efficient and methodologically robust under heterogeneous repository conditions. It demonstrates in practice that robust discovery, hierarchical normalization, and integrated temporal QA can be combined into a single coherent system without enterprise-scale infrastructure. Beyond the specific datasets evaluated, this matters because the same design pattern can be applied to other imaging cohorts where reproducibility, metadata integrity, and low-friction, iterative analysis are central requirements.

6.1. Future Work

Future development should prioritize three directions. First, testing and reproducibility should be strengthened through broader automated coverage and structured run manifests. Second, quality logic should become profile-driven so that different protocols can define explicit completeness and plausibility expectations. Third, semantic enrichment for private tags should be expanded through vendor-specific plugin mechanisms.

If deployment requirements grow, the architecture can evolve incrementally by introducing full-text indexing, background ingestion queues, and alternative storage backends while preserving existing module boundaries.

A. Reproducibility Commands

This appendix lists command templates only. Methodology, metric definitions, and interpretation are covered in Chapter 4. For portability and privacy, local absolute paths are replaced by placeholders. <RAW_DIR_X> and <FILTERED_DIR_X> denote dataset-specific input roots for dataset *X*; <DB_X> denotes an output databank path.

A.1. Filtered-Input Benchmark Commands (Primary Evaluation)

Use one command per dataset:

```
python3 process_dicom.py <FILTERED_DIR_X> <DB_X> --no-subdirs --timing --max-workers
    8
```

A.2. Three-Stage Size Measurement Commands

For each dataset, measure:

1. raw input folder size,
2. one-instance-per-series filtered folder size,
3. processed databank size.

One-instance-per-series subsets were generated with:

```
python3 extract_one_per_series.py <RAW_DIR_X> <ONE_PER_SERIES_DIR_X>
```

Size measurement (bytes):

```
du -sb <RAW_DIR_X> <ONE_PER_SERIES_DIR_X> <DB_X>
```

A.3. Raw vs Filtered Time and Memory Commands

```
/usr/bin/time -l python3 process_dicom.py <RAW_DIR_X> <RAW_DB_X> --no-subdirs --
    timing --max-workers 8
/usr/bin/time -l python3 process_dicom.py <FILTERED_DIR_X> <FILTERED_DB_X> --no-
    subdirs --timing --max-workers 8
```

A.4. Repeated Runtime Stability Commands

Run five repetitions per dataset:

```
python3 process_dicom.py <FILTERED_DIR_X> <REPEAT_DB_X> --no-subdirs --timing --no-
    auto-workers --max-workers 8
```

A.5. Automated Test Execution Commands

The implemented automated test suite can be executed locally with:

```
PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 python3 -m pytest tests
```

The suite currently contains 71 tests spanning discovery, extraction helpers, processing helpers, storage integration, study-service logic, dashboard-service logic, and anonymized export/web utility behavior.

A.6. Continuous Integration Configuration

Regression checks are also executed in GitHub Actions using:

```
.github/workflows/tests.yml
```

The workflow runs on push and pull-request events, installs project requirements, and executes the same regression suite through automated CI.

B. Supplementary Evaluation Notes

Values in Chapter 4 are environment-specific baselines and should be interpreted comparatively. For publication-grade benchmarking, keep controlled system load, fixed worker settings, and repeated measurements with variance reporting.

B.1. Hardware Specification

All benchmarks in this thesis were executed on the local development machine used throughout the project:

- Model: MacBook Pro (Model Identifier: Mac15,3)
- Chip: Apple M3
- CPU Cores: 8 total (4 performance + 4 efficiency)
- Memory: 8 GB
- Architecture: arm64
- Operating System: macOS 26.3 (Build 25D125)

B.2. Project Material on GitHub

All project materials used in this thesis are maintained in the GitHub repository:

`https://github.com/cccelik/DICOM-Metadata-Browser`

The repository contains:

- root entry points such as `process_dicom.py`, `extract\_one\_per\_series.py`, and `webui.py`,
- reusable application modules under `dicom_browser/`,
- the automated test suite under `tests/`,
- CI workflow definitions under `.github/workflows/`.
- the thesis report sources and generated document under `Efficient Data Access and Storage Optimization for PET:CT Imaging Data/`.

List of Figures

List of Tables

Bibliography

[1] National Electrical Manufacturers Association. *Digital Imaging and Communications in Medicine (DICOM) Standard*. https://www.dicomstandard.org/. Accessed 2026-02-27. 2024.

[2] DICOM Lookup. *DICOM Lookup Tag Browser*. http://www.dicomlookup.com/. Accessed 2026-03-01; used as practical tag lookup aid (non-normative). 2026.

[3] A. Fedorov, D. Clunie, E. Ulrich, C. Bauer, A. Wahle, B. Brown, M. Onken, J. Riesmeier, S. Pieper, R. Kikinis, J. Buatti, and R. R. Beichel. "DICOM for quantitative imaging biomarker development: a standards based approach to sharing clinical data and structured PET/CT analysis results in head and neck cancer research". In: *PeerJ* 4 (May 2016), e2057. DOI: 10.7717/peerj.2057.

[4] O. Diaz, K. Kushibar, R. Osuala, A. Linardos, L. Garrucho, L. Igual, P. Radeva, F. Prior, P. Gkontra, and K. Lekadir. "Data preparation for artificial intelligence in medical imaging: A comprehensive guide to open-access platforms and tools". In: *Physica Medica* 83 (2021), pp. 25–37. ISSN: 1120-1797. DOI: https://doi.org/10.1016/j.ejmp.2021.02.007. URL: https://www.sciencedirect.com/science/article/pii/S1120179721000958.

[5] A. Haidar, F. Aly, and L. Holloway. "PDCP: A Set of Tools for Extracting, Transforming, and Loading Radiotherapy Data from the Orthanc Research PACS". In: *Software* 1.2 (2022), pp. 215–222. ISSN: 2674-113X. DOI: 10.3390/software1020009. URL: https://www.mdpi.com/2674-113X/1/2/9.

[6] O. Pianykh. *Digital imaging and communications in medicine (DICOM): A practical introduction and survival guide*. Jan. 2008, pp. 1–383. ISBN: 9783540745709. DOI: 10.1007/978-3-540-74571-6.

[7] B. S. Erdal, L. M. Prevedello, S. Qian, M. Demirer, K. Little, J. Ryu, T. O'Donnell, and R. D. White. "Radiology and Enterprise Medical Imaging Extensions (REMIX)". In: *J Digit Imaging* 31.1 (Feb. 2018), pp. 91–106. DOI: 10.1007/s10278-017-0010-6.

[8] S. Jodogne. "The Orthanc Ecosystem for Medical Imaging". In: *Journal of Digital Imaging* 31.3 (2018), pp. 341–352. ISSN: 1618-727X. DOI: 10.1007/s10278-018-0082-y. URL: https://doi.org/10.1007/s10278-018-0082-y.

[9] F. Nagy, A. K. Krizsan, K. Kukuts, M. Szolikova, Z. Hascsi, S. Barna, A. Acs, P. Szabo, L. Tron, L. Balkay, M. Dahlbom, M. Zentai, A. Forgacs, and I. Garai. "Q-Bot: automatic DICOM metadata monitoring for the next level of quality management in nuclear medicine". In: *EJNMMI Phys* 8.1 (Mar. 2021), p. 28. DOI: 10.1186/s40658-021-00371-w.

[10] O. Tsave, A. Kosvyra, D. T. Filos, D. T. Fotopoulos, and I. Chouvarda. "A Multi-Dimensional Framework for Data Quality Assurance in Cancer Imaging Repositories". In: *Cancers* 17.19 (2025). ISSN: 2072-6694. DOI: 10.3390/cancers17193213. URL: https://www.mdpi.com/2072-6694/17/19/3213.

[11] L. Pei, G. Sutton, M. Rutherford, U. Wagner, T. Nolan, K. Smith, P. Farmer, P. Gu, A. Rana, K. Chen, T. Ferleman, B. Park, Y. Wu, J. Kojouharov, G. Singh, J. Lemon, T. Willis, M. Vukadinovic, G. Duffy, B. He, D. Ouyang, M. Pereanez, D. Samber, D. A. Smith, C. Cannistraci, Z. Fayad, D. S. Mendelson, M. Bufano, E. Kotter, H. Haghiri, R. Baidya, S. Dvoretskii, K. H. Maier-Hein, M. Nolden, C. Ablett, S. Siggillino, S. Kaushik, H. Jiang, S. Xie, Z. Wan, A. Michie, S. J. Doran, A. A. Waly, F. A. N. Liang, H. A. Mustagfirin, M. G. Felicia, K. P. Chih, R. Krish, G. Rasool, N. Bouaynaya, N. Koutsoubis, K. Naddeo, K. Pandit, T. O'Sullivan, R. Krish, Q. Pan, S. Gustafson, B. Kopchick, L. Opsahl-Ong, A. Olvera-Morales, J. Pinney, K. Johnson, T. Do, J. Klenk, M. Diaz, A. Singh, R. Chai, D. A. Clunie, F. Prior, and K. Farahani. *Medical Image De-Identification Benchmark Challenge*. 2025. arXiv: 2507.23608 [cs.CV]. URL: https://arxiv.org/abs/2507.23608.

[12] C. G. Schwarz, W. K. Kremers, T. M. Therneau, R. R. Sharp, J. L. Gunter, P. Vemuri, A. Arani, A. J. Spychalla, K. Kantarci, D. S. Knopman, R. C. Petersen, and C. R. Jack. "Identification of Anonymous MRI Research Participants with Face-Recognition Software". In: *New England Journal of Medicine* 381.17 (2019), pp. 1684–1686. DOI: 10.1056/NEJMc1908881. eprint: https://www.nejm.org/doi/pdf/10.1056/NEJMc1908881. URL: https://www.nejm.org/doi/full/10.1056/NEJMc1908881.

[13] R. Boellaard, R. Delgado-Bolton, W. J. Oyen, F. Giammarile, K. Tatsch, W. Eschner, F. J. Verzijlbergen, S. F. Barrington, L. C. Pike, W. A. Weber, S. Stroobants, D. Delbeke, K. J. Donohoe, S. Holbrook, M. M. Graham, G. Testanera, O. S. Hoekstra, J. Zijlstra, E. Visser, C. J. Hoekstra, J. Pruim, A. Willemsen, B. Arends, J. Kotzerke, A. Bockisch, T. Beyer, A. Chiti, B. J. Krause, and E. A. of Nuclear Medicine (EANM). "FDG PET/CT: EANM procedure guidelines for tumour imaging: version 2.0". In: *Eur J Nucl Med Mol Imaging* 42.2 (Feb. 2015). Epub 2014 Dec 2, pp. 328–54. DOI: 10.1007/s00259-014-2961-x.

[14] E. F. Codd. "A relational model of data for large shared data banks". In: *Commun. ACM* 13.6 (June 1970), pp. 377–387. ISSN: 0001-0782. DOI: 10.1145/362384.362685. URL: https://doi.org/10.1145/362384.362685.

[15] K. Jeblick, B. Schachtner, A. Mittermeier, J. Dexl, P. Wesp, T. Küstner, S. Gatidis, M. Früh, M. Fabritius, L. Unterrainer, G. Sheikh, A. Delker, G. Böning, M. Brendel, J. Ricke, R. Werner, S. Gu, M. Ingrisch, T. Geyer, and C. Cyran. *A whole-body PSMA-PET/CT dataset with manually annotated tumor lesions*. The Cancer Imaging Archive [dataset]. Version 2. 2026. DOI: 10.7937/r7ep-3x37. URL: https://doi.org/10.7937/r7ep-3x37.